

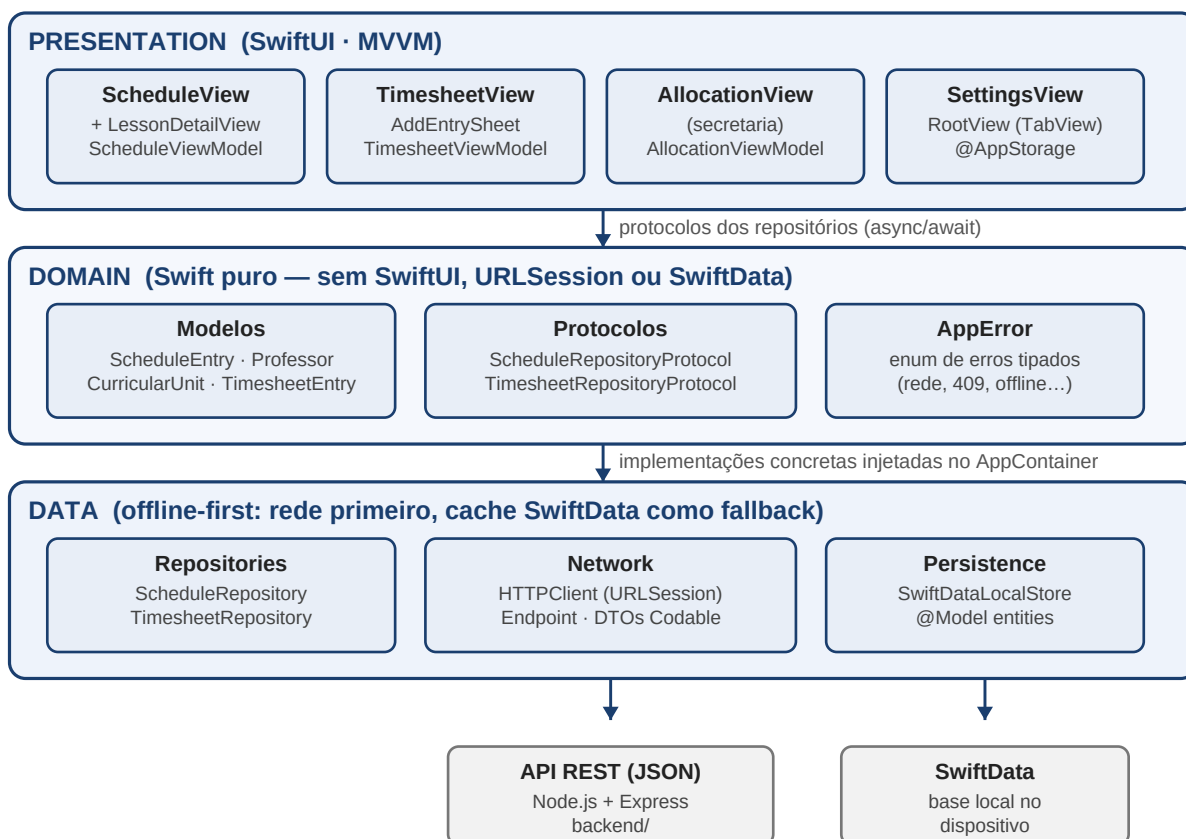
CampusHub — Documento de Arquitetura

Trabalho final · Disciplina de Desenvolvimento iOS · Mestrado em Dispositivos Móveis e Multimédia ·
Melissa Kaestner

1. Contexto e objetivo

O CampusHub centraliza três fluxos hoje informais numa instituição de ensino: a consulta da grelha horária (professores e alunos), o registo e resumo mensal de horas letivas (docentes pagos à hora) e a alocação de docentes a unidades curriculares pela secretaria, com validação automática de conflitos. A aplicação é nativa (Swift 5.9+, SwiftUI, iOS 17+), consome uma API REST própria (Node.js/Express) e funciona offline através de uma cache local em SwiftData.

2. Diagrama de camadas e módulos



A regra de dependência aponta sempre para o centro: **Presentation** conhece apenas os protocolos declarados no **Domain**; as implementações concretas vivem na camada **Data** e são compostas num único ponto (*AppContainer*, a raiz de composição), que as injeta por inicializador (*constructor injection*). O Domain é Swift puro e não importa SwiftUI, URLSession nem SwiftData.

3. Fluxo de dados (padrão offline-first)

1. O ViewModel coloca o ecrã em **.loading** e pede dados ao repositório via protocolo (async/await).
2. O repositório tenta a rede: o *HTTPClient* constrói o pedido a partir de um *Endpoint* declarativo e decodifica a resposta JSON para DTOs **Codable**, que são convertidos em modelos de domínio (entradas inválidas são descartadas em vez de provocar falha).
3. Em caso de sucesso, a cache SwiftData é **substituída** pelo conjunto recebido e os dados são devolvidos.
4. Em caso de falha de rede/servidor, o repositório devolve a cache local; se a cache estiver vazia, lança o erro tipado **offlineWithoutCache**.
5. O ViewModel traduz o resultado num estado explícito de UI — **loaded / empty / error(mensagem)** — e a View limita-se a renderizar esse estado (loading, lista, *empty state* ou erro com botão de repetição).

4. Decisões técnicas e justificações

Decisão	Justificação
MVVM com @Observable	ViewModels testáveis sem UI; a macro @Observable (iOS 17) reduz boilerplate face a ObservableObject e n
Protocolos para rede e persistência	HTTPClientProtocol e LocalStoreProtocol permitem substituir URLSession e SwiftData por mocks nos testes
Erros tipados (enum AppError)	Cada falha possível (rede, servidor, JSON inesperado, conflito 409, persistência, offline sem cache) tem um
DTOs separados do domínio	O formato do JSON remoto pode evoluir sem tocar no domínio nem na UI; a conversão DTO → domínio valid
SwiftData desnormalizado (snapshots)	SwiftData local é uma cache de leitura substituída integralmente a cada sincronização; sem escrita local diverge
Validação de conflitos no servidor	A detecção de sobreposição de horários (professor/sala) é regra de negócio central e deve valer para todos o

5. Estratégia de testes

Os testes (Swift Testing) cobrem as duas camadas exigidas. **Camada de dados:** os repositórios são testados com mocks do cliente HTTP e da persistência (sucesso remoto com atualização de cache, fallback offline, offline sem cache, conflito 409, descarte de dados inválidos); o SwiftDataLocalStore é testado contra um contentor SwiftData em memória (roundtrip, substituição de conjuntos, filtragem por professor). **ViewModels:** transições de estado (loading → loaded/empty/error), filtros e pesquisa do horário, cálculo do resumo mensal de horas, validação do formulário de alocação e mensagens de erro/sucesso. A injeção por inicializador garante que nenhum teste toca na rede ou no disco.